

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – Experiments

Course Code:	DSA0305	Course Title:	Natural Language Processing
CO Assessed:	CO3 – Precision	Assessment:	Assessment Tool 1 – Experiments
Weightage:	50% of CIA		
CO5 Statement:	Apply N-gram models, smoothing techniques, part-of-speech tagging systems and to evaluate effective syntactic analysis. (BL-3)		
Student Name:	A Roja	Reg. No.:	192424226
Section / Batch:	2024 / slot C	Date:	08-08-2026

Code of Conduct

I A Roja [192424226] (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: A Roja

Instructions

- Perform all experiments using Google Colab.
- Create a separate notebook (.ipynb) for the assessment.
- Ensure all code is executable without errors.
- Include appropriate comments explaining the logic used.
- Complete all three experiments in a single Google Colab notebook.
- Execute all cells and display the outputs for the given test cases.
- Save the notebook with the naming convention: RegNo_Name_CO3-AT1.ipynb
- Upload the notebook to your GitHub repository.
- Ensure the repository is public or accessible to the instructor.
- Submit the following:
- GitHub Repository Link in GCR
- Google Colab Share Link in GCR
- Screenshots of uploaded coding and output files as printout.
- Duration: 30 minutes.

Section / Topic	Focus	Experiment Questions
Context-Free Grammars for English Syntax, Context-Free Rules and Trees, Sentence-Level Constructions, Parsing, Top-Down Parsing, Earley Parsing	Design a Python program to validate sentences using CFG production rules and construct parse tree representations for valid sentences.	Q1
Agreement, Feature Structures, Sub Categorization	Design a Python program to verify subject-verb agreement and validate grammatical constraints using feature structures and subcategorization rules.	Q2
Probabilistic Context-Free Grammars (PCFGs)	Design a Python program to parse sentences and compute sentence probabilities using probabilistic grammar production rules.	Q3

Experiment 1: Context-Free Grammar Rule Validation and Parse Tree Construction

Experiment Description

Design a Python program to validate whether a sentence follows the given Context-Free Grammar (CFG) rules and generate its parse tree representation.

Grammar:

```
S → NP VP
NP → Det N
VP → V NP

Det → the | a
N → student | teacher | book
V → reads | likes
```

Task

1. Accept a sentence as input.
2. Verify whether it conforms to the CFG.
3. Construct and return the parse tree.
4. Return "Invalid Sentence" if parsing fails.

Function Prototype

```
def parse_cfg(sentence):
    pass
```

Test Cases

Input	Expected Output
the student reads a book	Valid Parse Tree
a teacher likes the book	Valid Parse Tree
student reads book	Invalid Sentence
the book likes a teacher	Valid Parse Tree
reads the student book	Invalid Sentence

Program:

```
def parse_cfg(sentence):
    words = sentence.lower().split()
    # Grammar
    determiners = {"the", "a"}
    nouns = {"student", "teacher", "book"}
    verbs = {"reads", "likes"}
    # Sentence must contain exactly 5 words
    if len(words) != 5:
        return "Invalid Sentence"
    det1, noun1, verb, det2, noun2 = words
    if det1 not in determiners:
        return "Invalid Sentence"
    if noun1 not in nouns:
        return "Invalid Sentence"
    if verb not in verbs:
```

```

        return "Invalid Sentence"
    if det2 not in determiners:
        return "Invalid Sentence"
    if noun2 not in nouns:
        return "Invalid Sentence"
    parse_tree = f"""
S
├── NP
│   ├── Det → {det1}
│   └── N → {noun1}
└── VP
    ├── V → {verb}
    └── NP
        ├── Det → {det2}
        └── N → {noun2}
"""

    return "Valid Parse Tree:" + parse_tree
sentences = [
    "the student reads a book",
    "a teacher likes the book",
    "student reads book",
    "the book likes a teacher",
    "reads the student book"
]
for sentence in sentences:
    print("Input:", sentence)
    print(parse_cfg(sentence))
    print("-" * 40)

```

OUTPUT:

```

File Edit Format Run Options Window Help
# Experiment 1: CFG Rule Validation and Parse Tree Construction

def parse_cfg(sentence):
    words = sentence.lower().split()

    # Grammar
    determiners = {"the", "a"}
    nouns = {"student", "teacher", "book"}
    verbs = {"reads", "likes"}

    # Sentence must contain exactly 5 words
    # S -> NP VP
    # NP -> Det N
    # VP -> V NP
    if len(words) != 5:
        return "Invalid Sentence"

    det1, noun1, verb, det2, noun2 = words

    # Check grammar rules
    if det1 not in determiners:
        return "Invalid Sentence"

    if noun1 not in nouns:
        return "Invalid Sentence"

    if verb not in verbs:
        return "Invalid Sentence"

    if det2 not in determiners:
        return "Invalid Sentence"

    if noun2 not in nouns:
        return "Invalid Sentence"

    # Construct parse tree
    parse_tree = f"""
S
├── NP
│   ├── Det → {det1}
│   └── N → {noun1}
└── VP
    ├── V → {verb}
    └── NP
        ├── Det → {det2}
        └── N → {noun2}
"""

    return "Valid Parse Tree:" + parse_tree

sentences = [
    "the student reads a book",
    "a teacher likes the book",
    "student reads book",
    "the book likes a teacher",
    "reads the student book"
]
for sentence in sentences:
    print("Input:", sentence)
    print(parse_cfg(sentence))
    print("-" * 40)

```

```

Python 3.13.3 (tags/v3.13.3:6280bb5, Apr 8 2025, 14:47:33) [MSC v.
AMD64] on win32
Enter "help" below or click "Help" above for more information.

>>>
V/Downloads/ass 1.py ----- RESTART: C:/U
Input: the student reads a book
Valid Parse Tree:
S
├── NP
│   ├── Det → the
│   └── N → student
└── VP
    ├── V → reads
    └── NP
        ├── Det → a
        └── N → book

-----
Input: a teacher likes the book
Valid Parse Tree:
S
├── NP
│   ├── Det → a
│   └── N → teacher
└── VP
    ├── V → likes
    └── NP
        ├── Det → the
        └── N → book

-----
Input: student reads book
Invalid Sentence

-----
Input: the book likes a teacher
Valid Parse Tree:
S
├── NP
│   ├── Det → the

```



```

"runs": {"number": "singular"},
"writes": {"number": "singular"},
"run": {"number": "plural"},
"write": {"number": "plural"}

```

```

subject = subject.lower()
verb = verb.lower()
if subject not in subjects or verb not in verbs:
    return False
if subjects[subject]["number"] == verbs[verb]["number"]:
    return True
else:
    return False
# Test Cases
test_cases = [
    ("he", "runs"),
    ("he", "run"),
    ("they", "run"),
    ("they", "writes"),
    ("she", "writes")
]
for subject, verb in test_cases:
    result = check_agreement(subject, verb)
    print("Subject:", subject)
    print("Verb:", verb)
    print("Agreement:", result)
    print("-" * 30)

```

OUTPUT:

The screenshot shows a Python script in the left pane and its execution output in the IDLE Shell on the right. The script defines subject and verb features and checks for agreement between them. The output shows the results for five test cases, with agreement being True for 'he runs', 'they run', and 'she writes', and False for 'he run' and 'they writes'.

```

File Edit Format Run Options Window Help
"""
it": {"number": "singular", "person": "third"},
they": {"number": "plural", "person": "third"}
}

# Feature structures for verbs
verbs = {
    "runs": {"number": "singular"},
    "writes": {"number": "singular"},
    "run": {"number": "plural"},
    "write": {"number": "plural"}
}

subject = subject.lower()
verb = verb.lower()

# Check whether subject and verb exist
if subject not in subjects or verb not in verbs:
    return False

# Compare Number feature
if subjects[subject]["number"] == verbs[verb]["number"]:
    return True
else:
    return False

# Test Cases
test_cases = [
    ("he", "runs"),
    ("he", "run"),
    ("they", "run"),
    ("they", "writes"),
    ("she", "writes")
]

for subject, verb in test_cases:
    result = check_agreement(subject, verb)
    print("Subject:", subject)
    print("Verb:", verb)
    print("Agreement:", result)
    print("-" * 30)

```

```

IDLE Shell 3.13.3
File Edit Shell Debug Options Window Help
Python 3.13.3 (tags/v3.13.3:6280bb5, Apr 8 2025, 14:47:33) [MSC v.1943
AMD64] on win32
Enter "help" below or click "Help" above for more information.

>>> ===== RESTART: C:/Users/ROJAYADAV/Downloads/ass 1.py =====
Subject: he
Verb: runs
Agreement: True
-----
Subject: he
Verb: run
Agreement: False
-----
Subject: they
Verb: run
Agreement: True
-----
Subject: they
Verb: writes
Agreement: False
-----
Subject: she
Verb: writes
Agreement: True
-----
>>> |

```

Experiment 3: Probabilistic Context-Free Grammar (PCFG) Parsing

Experiment Description

Implement a Python program to compute the probability of a sentence using the given PCFG.

Grammar

$S \rightarrow NP \ VP \quad [1.0]$

$NP \rightarrow John \quad [0.6]$

$NP \rightarrow Mary \quad [0.4]$

$VP \rightarrow runs \quad [0.5]$

$VP \rightarrow walks \quad [0.5]$

Probability Formula

$P(\text{Sentence})$

$=$

$P(S \rightarrow NP \ VP)$

$\times P(NP)$

$\times P(VP)$

Task

1. Accept a two-word sentence.
2. Parse using the PCFG.
3. Compute sentence probability.
4. Return 0 if sentence cannot be generated.

Function Prototype

```
def sentence_probability(sentence):  
    pass
```

Test Cases

Input Sentence Expected Probability

John runs	0.30
John walks	0.30
Mary runs	0.20
Mary walks	0.20
Peter runs	0.00

PROGRAM:

```
def sentence_probability(sentence):  
    words = sentence.lower().split()  
    if len(words) != 2:  
        return 0.0  
    subject = words[0]  
    verb = words[1]  
    # PCFG probabilities  
    np_prob = {  
        "john": 0.6,  
        "mary": 0.4  
    }  
    vp_prob = {  
        "runs": 0.5,  
        "walks": 0.5  
    }  
    s_prob = 1.0  
    if subject not in np_prob:  
        return 0.0  
    if verb not in vp_prob:  
        return 0.0
```

```

probability = (
    s_prob
    * np_prob[subject]
    * vp_prob[verb]
)
return probability
sentences = [
    "John runs",
    "John walks",
    "Mary runs",
    "Mary walks",
    "Peter runs"
]
for sentence in sentences:
    probability = sentence_probability(sentence)
    print("Input:", sentence)
    print("Probability:", f"{probability:.2f}")
    print("-" * 30)

```

OUTPUT

```

File Edit Format Run Options Window Help

vp_prob = {
    "runs": 0.5,
    "walks": 0.5
}

# Probability of S -> NP VP
s_prob = 1.0

# Check whether words can be generated
if subject not in np_prob:
    return 0.0

if verb not in vp_prob:
    return 0.0

# Calculate sentence probability
probability = (
    s_prob
    * np_prob[subject]
    * vp_prob[verb]
)

return probability

# Test Cases
sentences = [
    "John runs",
    "John walks",
    "Mary runs",
    "Mary walks",
    "Peter runs"
]

for sentence in sentences:
    probability = sentence_probability(sentence)
    print("Input:", sentence)
    print("Probability:", f"{probability:.2f}")
    print("-" * 30)

```

```

IDLE Shell 3.13.3
File Edit Shell Debug Options Window Help
Python 3.13.3 (tags/v3.13.3:6280bb5, Apr
AMD64) on win32
Enter "help" below or click "Help" above f
>>>
===== RESTART: C:/Users/ROJAYAI
Input: John runs
Probability: 0.30
-----
Input: John walks
Probability: 0.30
-----
Input: Mary runs
Probability: 0.20
-----
Input: Mary walks
Probability: 0.20
-----
Input: Peter runs
Probability: 0.00
-----
>>>

```

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Q. No. / Criteria Level	Understanding of Concepts	Experimental Design	Use of Tools	Analysis of Results	Documentation	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____